Assignment 3 - Tiled Matrix Multiplication

> DUE Monday, October 26 @ 11:59:59PM Pacific Time

Overview

The objective of this assignment is to implement a *tiled* matrix multiplication kernel that can support arbitrary sized matrices.

Tiled Matrix Multiplication

1. For this lab, we will be using Classroom 50 (replacement for GitHub Classroom).

   > Please join the classroom assignment by clicking the following link: https://classroom50.org/UCR-CSEE217/cs-ee217-fall-2026/assignments/matrix-multiply/accept

   Once you accept the assignment, a private GitHub repository will automatically be created with the starter code. Simply `git clone` it to copy the starter code to Bender.

2. Edit the source files `kernel.cu` and `main.cu` to complete the functionality of the matrix multiplication on the device. Fill in the `INSERT CODE HERE` sections: allocating device memory, copying data to and from the GPU, freeing memory (`main.cu`),

and the kernel and its launch (`kernel.cu`). `support.cu`, `support.h` and the `Makefile` are provided; do not modify them. Wrap CUDA API calls in `CHECK(...)`, e.g. `CHECK(cudaMalloc(...))`, so errors are reported with the line they came from.

> The two matrices could be any size, but we will not test your code with an output matrix size exceeding 65,536 elements (for example, 256 x 256 input matrices) in GPGPU-Sim. This is purely a limitation for testing your code in a timely manner. Your code should still be able to run for significantly larger matrices, and the automated tests use sizes up to about 1500.

Size your thread blocks with the provided `BLOCK_X` and `BLOCK_Y` (16 x 16 by default; see Changing the matrix size and block size). Your code must work for any matrix shape and any tile size, not just square matrices that are a multiple of the tile.

3. There are three modes of operation for the application. Check `main()` for a description of the modes (repeated below). You will support each of these modes using a Tiled matrix multiplication implementation.

   - **No arguments**: The application will create two randomly initialized matrices to multiply size (1000x1000). After the device multiplication is invoked, it will compute the correct solution matrix using the CPU, and compare that solution with the device-computed solution. If it matches (within a small tolerance), it will print out "TEST PASSED" to the screen before exiting.
   - **One argument**: The application will use the random initialization to create the input matrices (size mxm, where m is the argument. Start your testing with small matrices.
   - **Three arguments** m, k, and n: The application will initialize the two input matrices with random values. A matrix will be of size m x k while the B matrix will be of size k x n, producing a C matrix of size m x n

4. Build and test on Bender:

```
make
./sgemm-tiled                 # 1000 x 1000 matrices
```

```
./sgemm-tiled 128              # 128 x 128
./sgemm-tiled 120 201 250      # A: 120 x 201, B: 201 x 250
```

The program checks every element of your result against the CPU and prints `TEST PASSED`, or lists the first wrong elements. Bender's CUDA tools may not be on your PATH; if `nvcc` is not found, add `export PATH=/usr/local/cuda/bin:$PATH` to `~/.bashrc`. To check for problems that can go unnoticed otherwise, run your program under `compute-sanitizer`:

```
compute-sanitizer ./sgemm-tiled 33 17 65                    # out-of-bounds accesses
compute-sanitizer --tool racecheck ./sgemm-tiled 33 17 65   # missing __syncthreads()
compute-sanitizer --tool synccheck ./sgemm-tiled 33 17 65   # __syncthreads() in a divergent branch
compute-sanitizer --tool initcheck ./sgemm-tiled 33 17 65   # uninitialized memory
```

5. Commit and push your completed tiled matrix multiplication code to the private repository.

Changing the matrix size and block size

Matrix size is set on the command line when you run the program; no rebuild is needed:

Command	Matrices
`./sgemm-tiled`	1000 x 1000 (default)
`./sgemm-tiled 512`	512 x 512
`./sgemm-tiled 120 201 250`	A: 120 x 201, B: 201 x 250, C: 120 x 250

Block size (the tile size) is set when you build. `basicSgemm()` in `kernel.cu` provides `BLOCK_X` and `BLOCK_Y`, the number of threads per block in x and y (16 x 16 by default), and `TILE_X` and `TILE_Y` are available for sizing your shared-memory tiles. Use them rather than hard-coded numbers; otherwise changing the block size has no effect. `make` rebuilds automatically whenever a block-size setting changes.

- **Square blocks:** set both dimensions with `TILE`:

```
make TILE=8     # 8 x 8 blocks
./sgemm-tiled 256
```

- **Non-square blocks:** set each dimension with `TILE_X` and `TILE_Y`:

```
make TILE_X=32 TILE_Y=8    # 32 x 8 blocks
./sgemm-tiled 256
```

- **Back to the default:** plain `make` builds with 16 x 16 blocks again.

Automated tests

Every time you push to `main`, GitHub automatically builds your code and runs a series of tests. Open the **Actions** tab of your repository to see the results. The tests cover:

- Many matrix sizes: tiny and single-row or single-column matrices, sizes just below, at and above the tile size, odd rectangular shapes, and larger matrices.
- Other block sizes and shapes: the code is rebuilt with `TILE=4`, `8`, `32` and with non-square blocks (32x8, 8x32, 4x16, 16x4, 16x64, 64x16, 8x128, 128x8), so do not hard-code 16.

- `compute-sanitizer` memory, race, synchronization and uninitialized-memory checks.

If a test fails, click on it: the summary explains the cause (wrong results, a CUDA error, an invalid memory access, a shared-memory race, an invalid `__syncthreads()`, uninitialized memory, a build failure or a timeout), and the log shows your program's full output.

GPGPU-Sim analysis

> Please see the GPGPU-Sim setup guide.

We will now analyze memory behavior of tiled matrix multiplication using GPGPU-Sim.
To aid in analyzing the microarchitectural properties of these programs, it may help to save the output of the GPGPU-Sim run into a file. You may save the output by redirecting the printouts to a file using `./sgemm-tiled &> outfile`. Do NOT commit any output files to the git repository.

Since the focus of this lab is on memory performance, we will focus mainly on these memory statistics:

```
gpgpu_n_load_insn   Number of global/local load instructions executed.
gpgpu_n_store_insn  Number of global/local store instructions executed.
gpgpu_n_shmem_insn  Number of shared memory instructions executed.
```

To compare run times, use `gpu_tot_sim_cycle`, the total number of simulated GPU cycles. Simulating is slow, so use a 128 x 128 matrix (`./sgemm-tiled 128`) for the tile-size comparison; it takes about a minute and a half at most.

Keep in mind what hardware is being simulated: GPGPU-Sim is configured to model an NVIDIA **GTX480**, whose streaming multiprocessors (SMs) can each hold at most **1536 threads** at a time. Think about how the block size you choose affects how many threads fit on an SM, and how that trades off against global memory accesses.

Report

Answer the questions in `REPORT.md` in your repository. Expand "How to use this file" at the top of it for how to add plots, tables and math. Some questions ask you to measure and plot results from your program and from GPGPU-Sim. You collect the numbers and make the plots yourself, with any tool you like. Save the plots as PNG files in `plots/` and embed them in `REPORT.md`. Do NOT commit GPGPU-Sim output files.

Some questions ask you to try other tile sizes and shapes; see Changing the matrix size and block size. **Submit your code with the default 16x16 blocks**: don't change the `TILE_SIZE`, `TILE_X` or `TILE_Y` defaults in `kernel.cu`. The automated tests start from a plain `make`.

Submission

1. Fill in your name and answers in `REPORT.md`, and add your plots to `plots/`.
2. Commit and push your completed tiled matrix multiplication code (`kernel.cu` and `main.cu`), `REPORT.md` and `plots/` to the github repository. Open `REPORT.md` on GitHub to check that your plots display; that's the version we grade.
3. Check that the automated tests pass on your final commit.

© 2019 - 2026 Daniel Wong